FACULTY OF COMPUTING, INFORMATICS AND MATHEMATICS UNIVERSITY TEKNOLOGI MARA (UITM) SHAH ALAM, SELANGOR

BACHELOR OF INFORMATION SYSTEM (HONS.) COMPUTER SCIENCE

CSC584 ENTERPRISE PROGRAMMING

GROUP PROJECT

TITLE: UniVelo UiTM Ride-Hailing

PREPARED BY

GROUP	STUDENT ID	NAME
CS2304E	2025239766	MUHAMMAD SYAZWAN BIN MAHAD
CS2304E	2025240964	MUHAMMAD IZWAN BIN MOHD SHARIL
CS2304E	2025478438	MUHAMMAD NAFIS ZAHIN BIN SHAHRUL AZAM
CS2304E	2025231864	MUHAMMAD AZREEZ BIN AHMAD ZAMZURI

PREPARED FOR:

DR. SHAKIRAH BINTI HASHIM

TABLE OF CONTENTS

1.0 INTRODUCTION

The UniVelo: UiTM E-Hailing System is a centralized, web-based platform designed to streamline campus mobility and transportation management across the university. By leveraging Java web technology and the MVC architecture, this system ensures scalability, reliability, and ease of use for all users. It enables students to easily request rides, track vehicle locations in real-time, and manage their daily transit, while providing administrators and drivers with robust tools to organize rides, handle logistics, and monitor campus traffic efficiently. With a focus on user experience and campus safety, the system aims to bring all aspects of university transit engagement into one centralized platform, improving communication, efficiency, and overall connectivity across the UiTM campus.

2.0 PROBLEM STATEMENT

- Campus mobility and transportation services are currently managed separately across different platforms or communicated manually (e.g., via informal chat groups), leading to inefficient coordination.

- Students find it difficult to track available drivers in real-time, view open ride options, and keep an accurate record of their own past transit history.

- The absence of a centralized campus e-hailing system creates difficulties for students trying to request rides, drivers trying to update their availability, and administrators attempting to monitor overall transit logistics and campus travel safely.

3.0 OBJECTIVE

- To develop a centralized web-based system for managing campus mobility and transportation across the university.
- To allow students to view and request rides from all available drivers in a simple and organized manner.
- To record and track student transit history and real-time ride locations accurately, ensuring up-to-date trip records.
- To enable drivers and administrators to manage ride requests, update vehicle availability, and monitor transit logistics easily, streamlining campus travel and communication.

4.0 ENTITY RELATIONAL DIAGRAM (ERD)

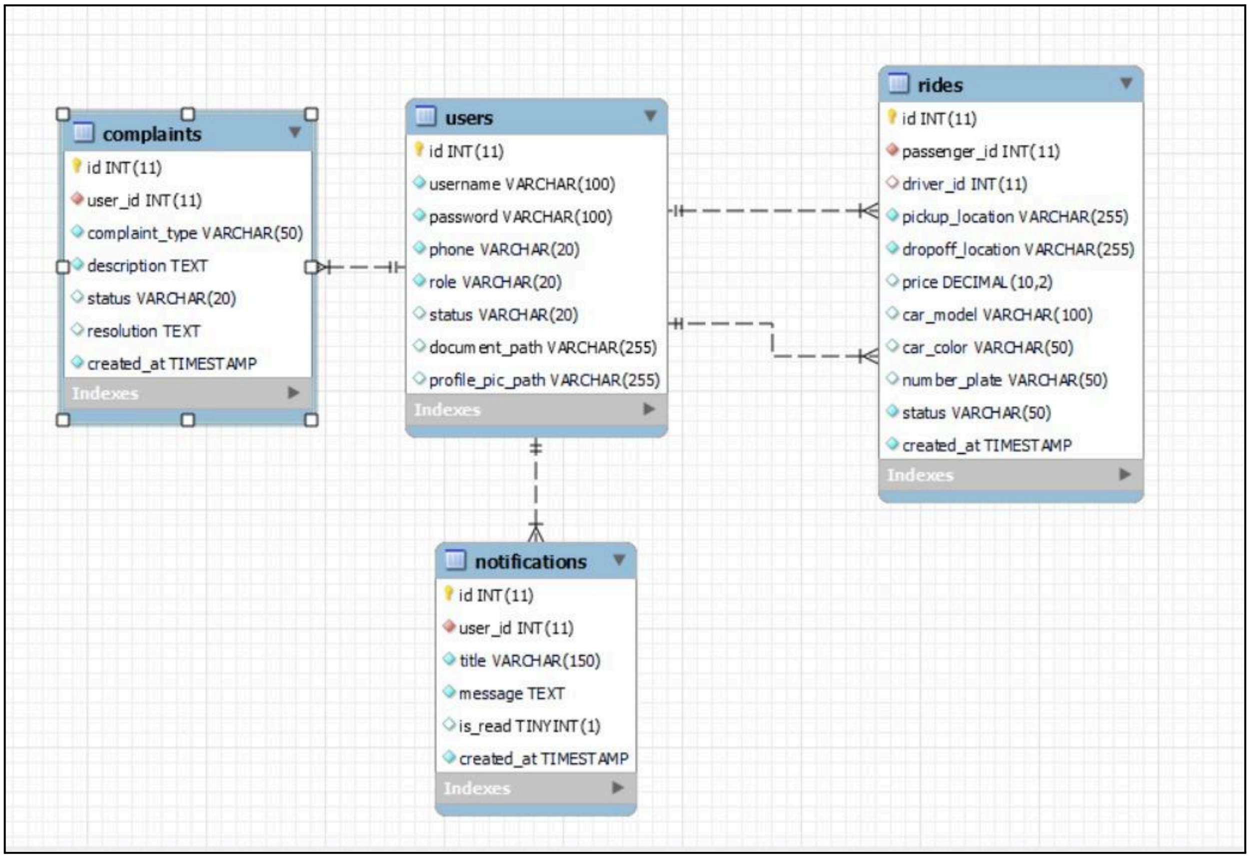

Figure 4.1 Entity Relational Diagram

The Entity Relationship Diagram (ERD) for Univelo outlines a centralized database structure designed for a ride-sharing system where the `users` table serves as the primary hub. This core entity holds profile and role information (such as passenger or driver status) and connects to all other tables in a one-to-many relationship. Specifically, it links to `rides` to track trip details, mapping both a `passenger_id` and a `driver_id` to individual journeys, while also connecting to `complaints` for logging user support tickets and `notifications` for pushing updates and system alerts to individual accounts.

5.0 NETBEANS FILE STRUCTURE

5.1 JSP FILES

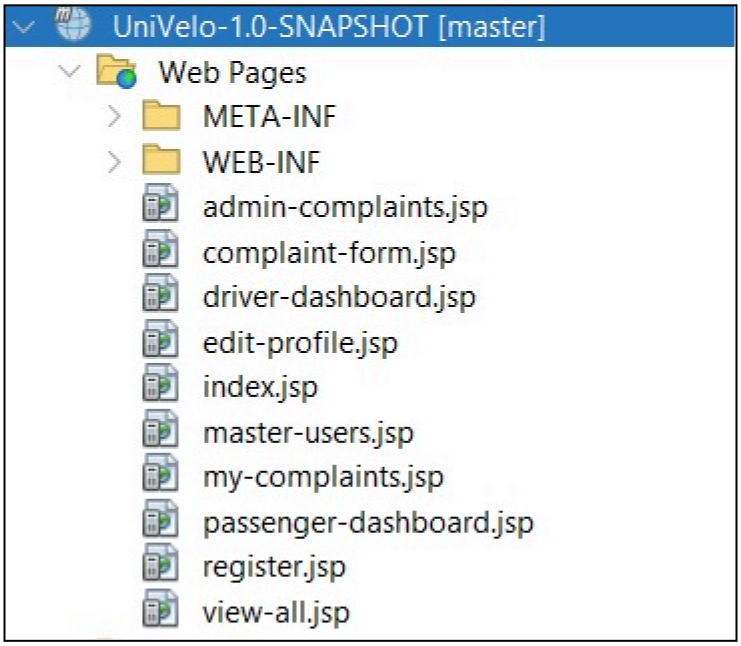

Figure 5.1.1 JSP Files

Figure 5.1.1 shows all the JSP files or interfaces that are available in UniVelo including all the present users such as admin, passenger and driver. There are also interfaces for all the functions that are available in UniVelo.

5.2 SERVLET / CONTROLLER FILES

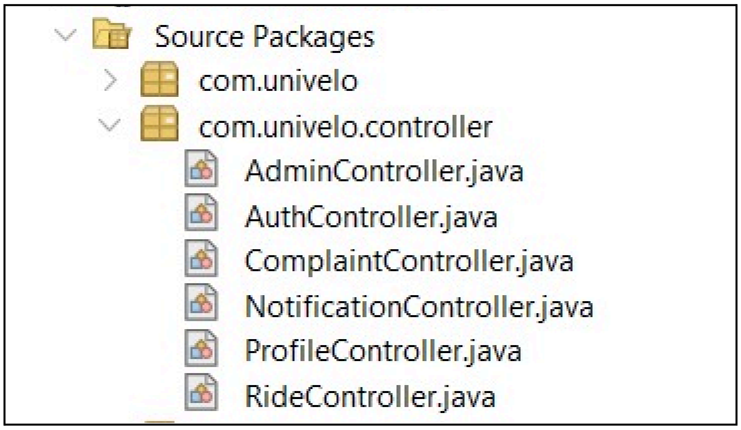

Figure 5.2.1 Servlet Files

Figure 5.2.1 shows all the servlet files that are implemented in UniVelo. It covers all the functions logic in the system.

5.3 DATA ACCESS OBJECT FILES

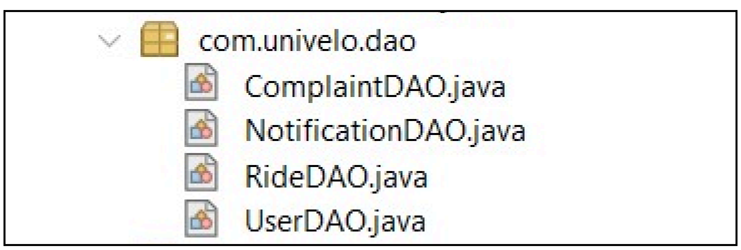

Figure 5.3.1 DAO Files

Figure 5.3.1 shows all the data access object files that are implemented in UniVelo. It covers all the database functions in UniVelo.

5.4 JAVABEANS / MODEL FILES

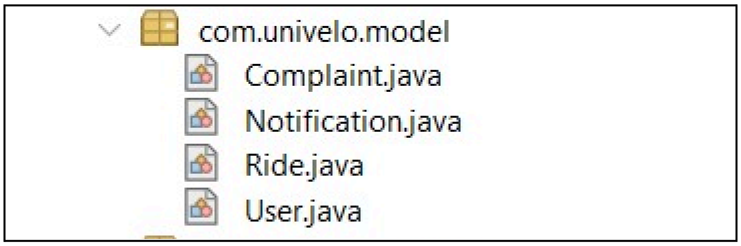

Figure 5.4.1 Model Files

Figure 5.4.1 shows all the JavaBeans / Model files. These files represent the data layer of the UniVelo system, where each Java class corresponds directly to a database table. They are used to encapsulate, store, and transfer data between the database and the user interface.

USER MANUAL

UniVelo UiTM Ride-Hailing

6.1 SIGN UP & REGISTRATION

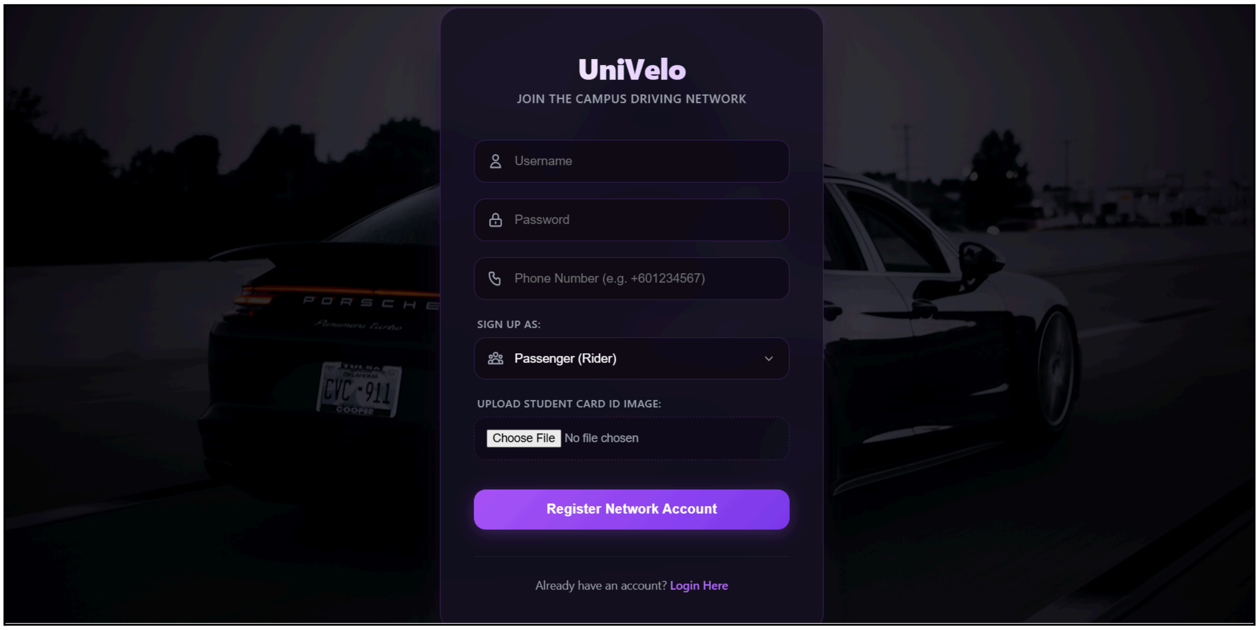

Figure 6.1.1 Sign Up Page

Figure 6.1.1 shows the Sign Up page for all users (admin / driver / passanger).

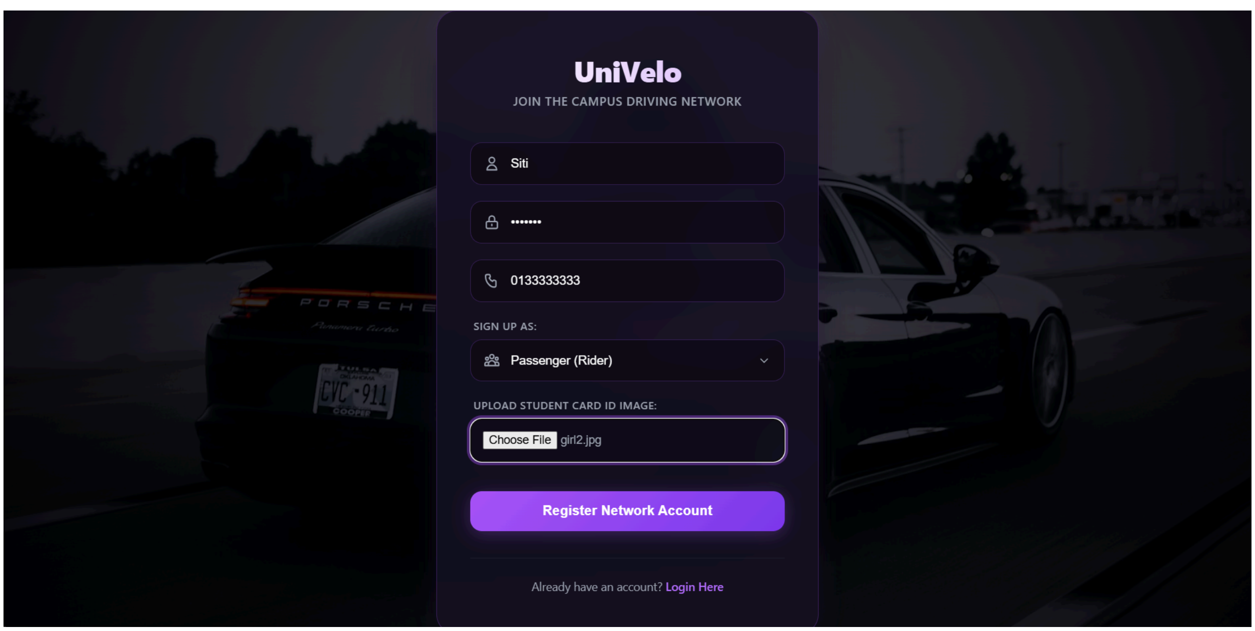

Figure 6.1.2 Filled In Sign Up Page

Figure 6.1.2 shows the user filling in their profile information, select a role (Passenger or Driver), and upload the required credential:

● **Passengers:** Must upload a valid Student Card.
● **Drivers:** Must upload a valid Driver's License.

Submission: Clicking the **Register** button submits the application to the Admin for verification.

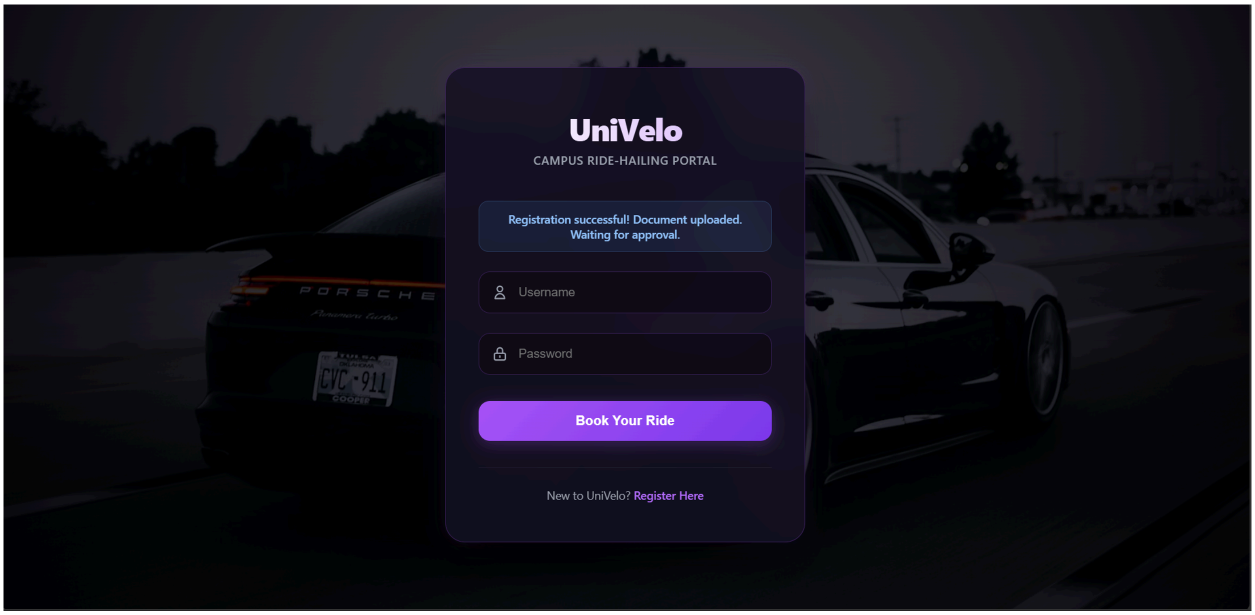

Figure 6.1.3 Successful Registration Notification

Figure 6.1.3 shows a confirmation pop-up appears immediately after a successful submission.

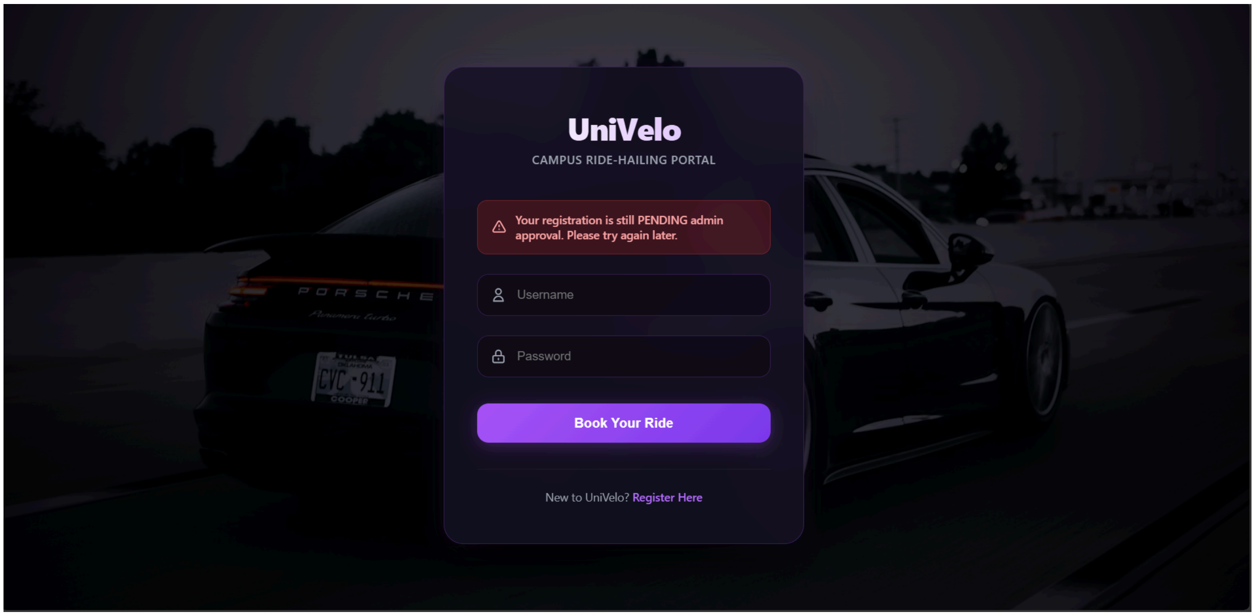

Figure 6.1.4 Pending Notification

Figure 6.1.4 shows a **PENDING** notification on the user's dashboard if the Admin has not yet reviewed the application.

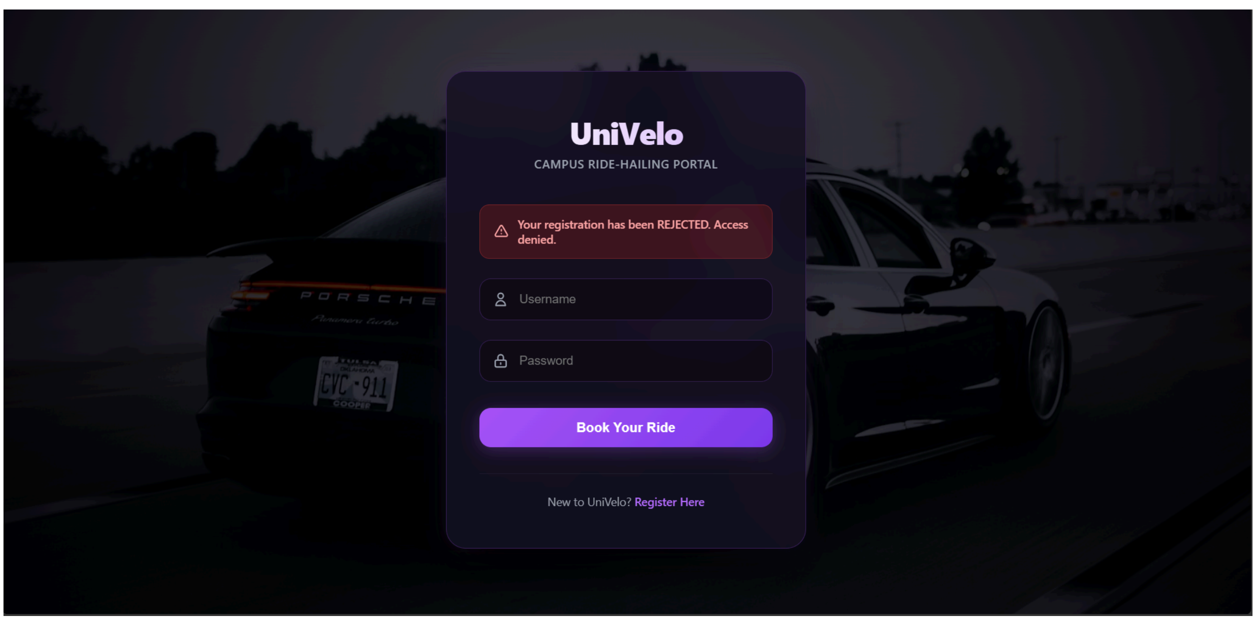

Figure 6.1.5 Rejected Notification

Figure 6.1.4 shows a **REJECTED** notification if the Admin denies the application.

6.2 ADMIN

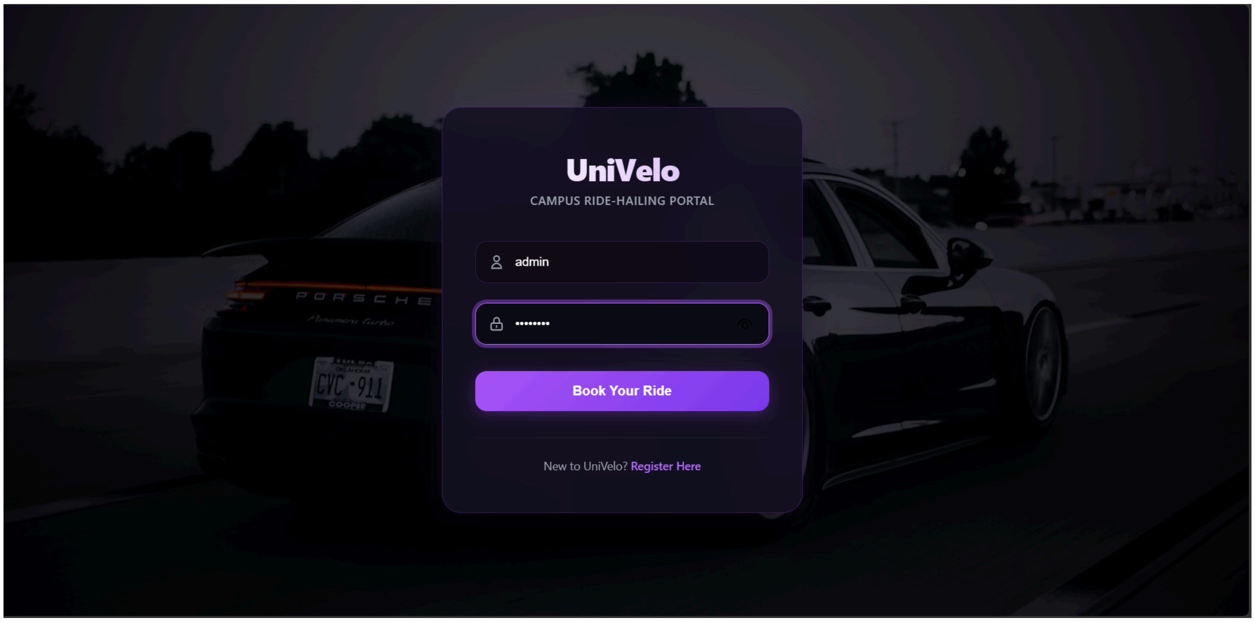

Figure 6.2.1 Login Page with Admin's ID

Figure 6.2.1 shows a secure portal for approved admins.

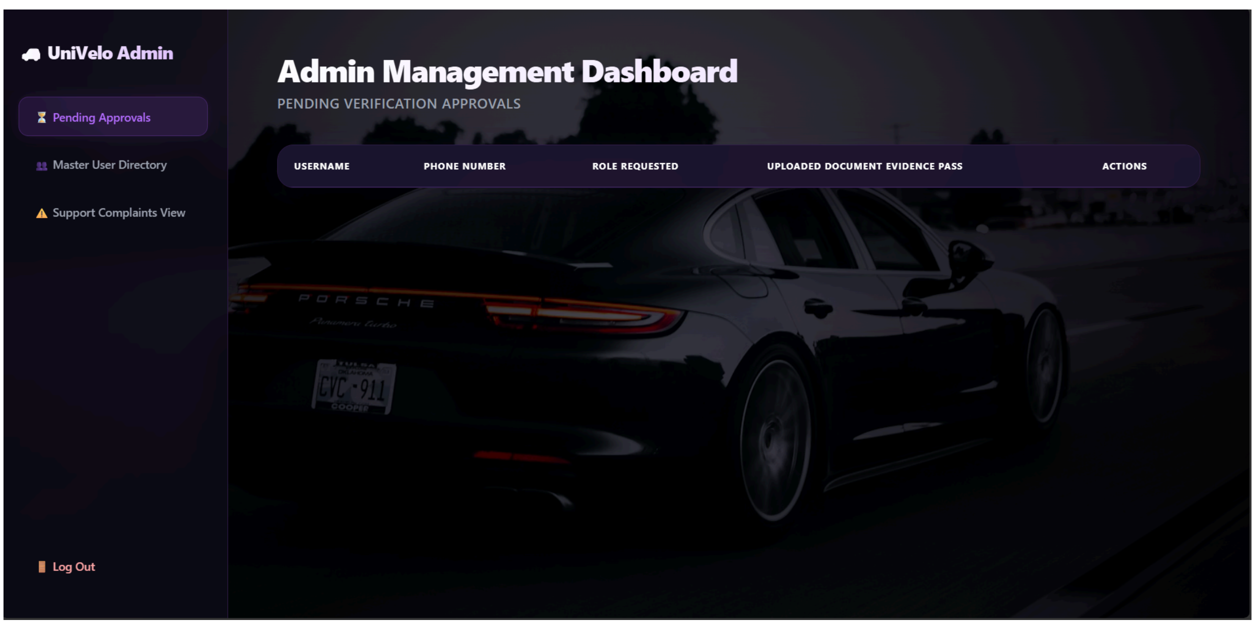

Figure 6.2.2 Admin Management Dashboard

Figure 6.2.2 shows the basic interface for admin.

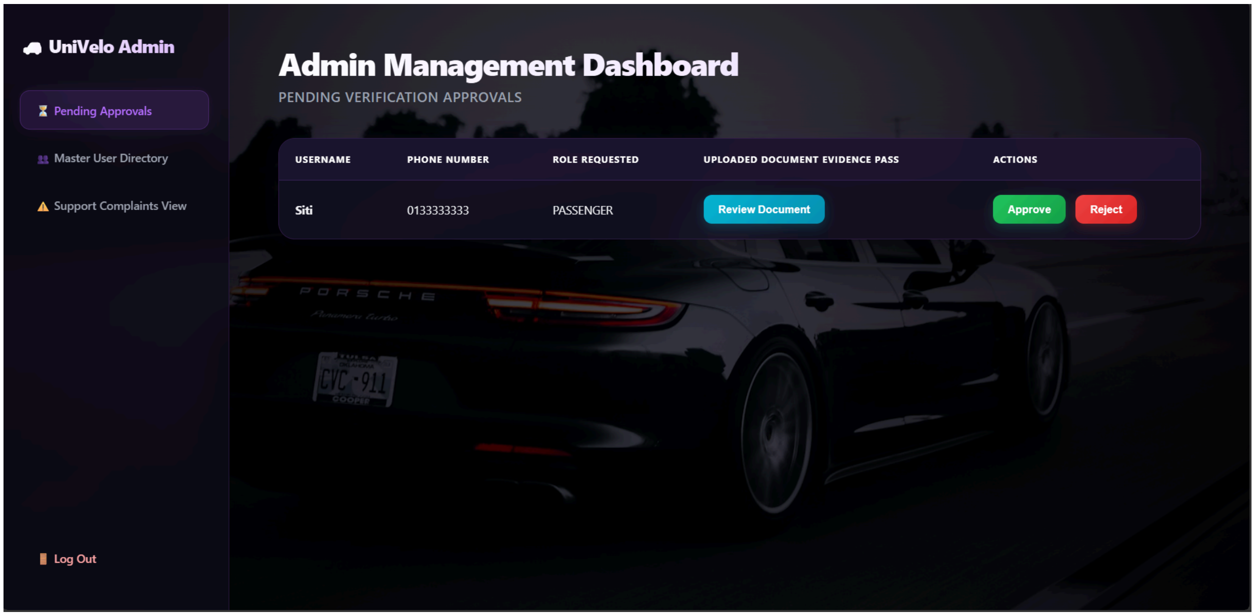

Figure 6.2.3 Admin Management Dashboard with Pending User Application

Figure 6.2.3 shows all incoming user registration applications. Admins can review uploaded documents (Student Cards or Driver's Licenses) to either **Approve** or **Reject** the applicant.

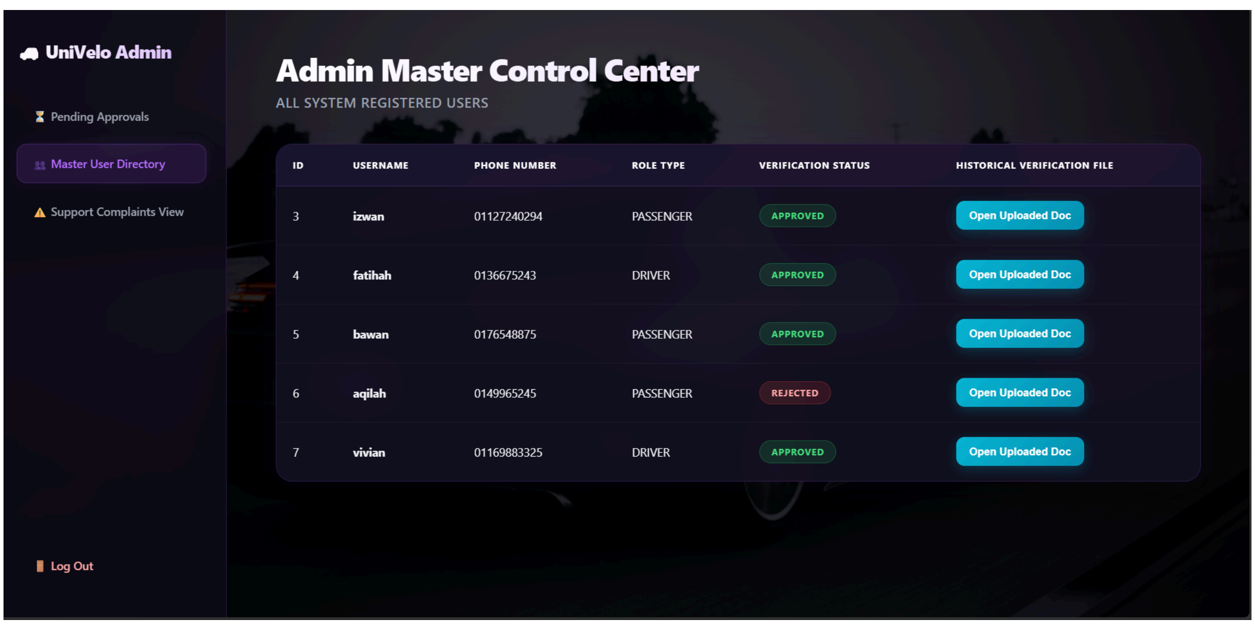

Figure 6.2.4 Admin Master Control Centre

Figure 6.2.4 shows a comprehensive overview of all registered users in the system, including the ability to view their previously verified documents.

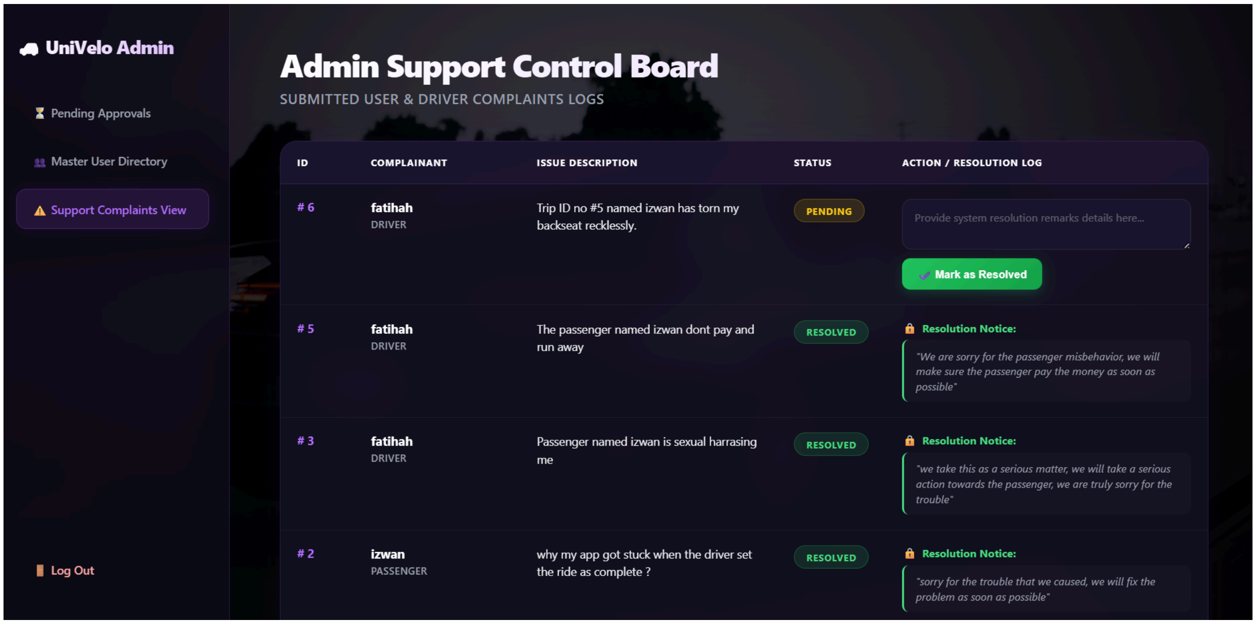

Figure 6.2.5 Admin Support Control Board

Figure 6.2.5 shows a centralized hub for managing user complaints. Admins can view passenger and driver complaints, log resolution details, send a **Resolution Notice** to the reporter, and mark issues as **Resolved**.

6.3 DRIVER

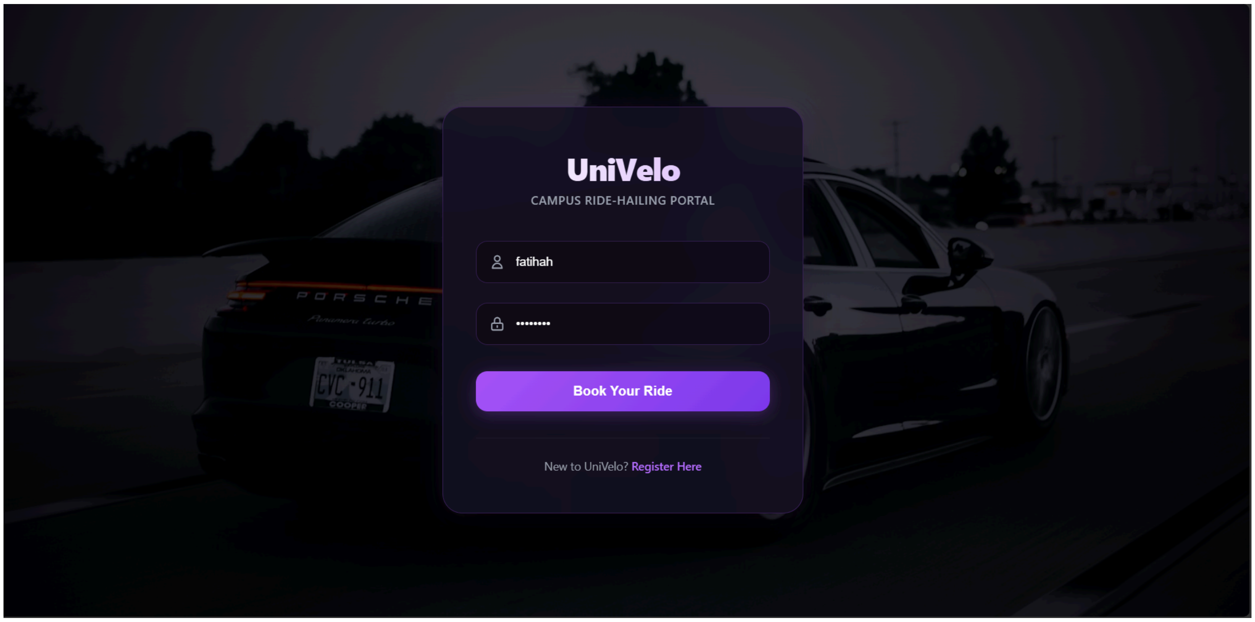

Figure 6.3.1 Login Page with Driver's ID

Figure 6.3.1 shows a secure portal for approved drivers.

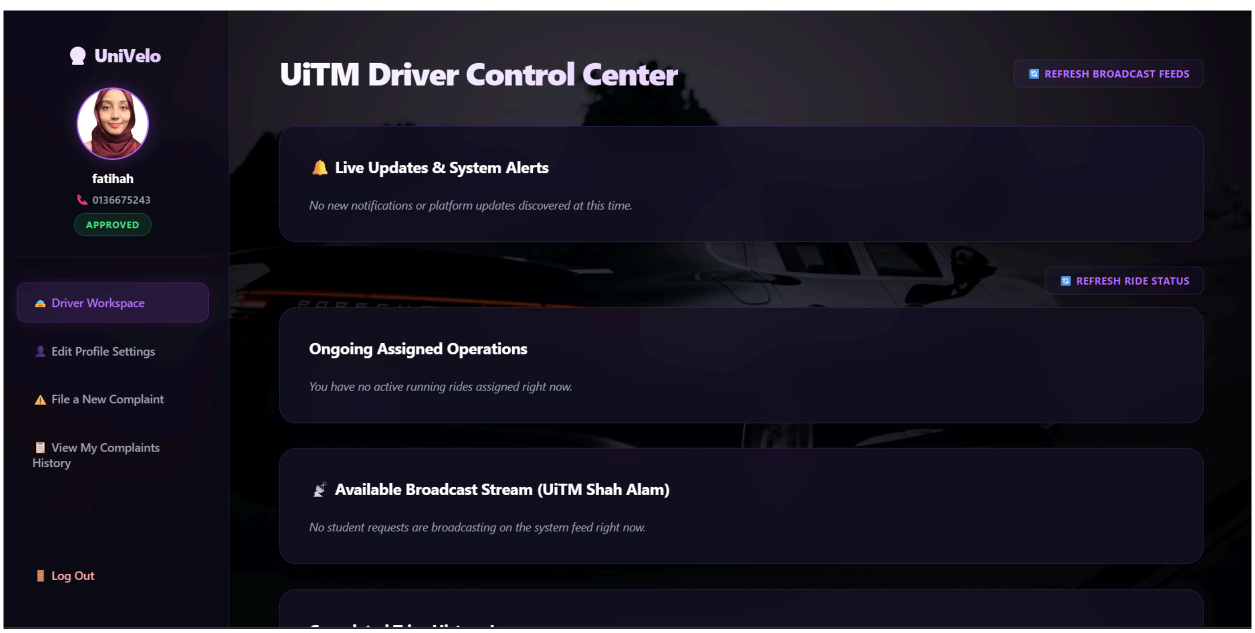

Figure 6.3.2 Driver Control Centre

Figure 6.3.2 shows the basic interface for drivers.

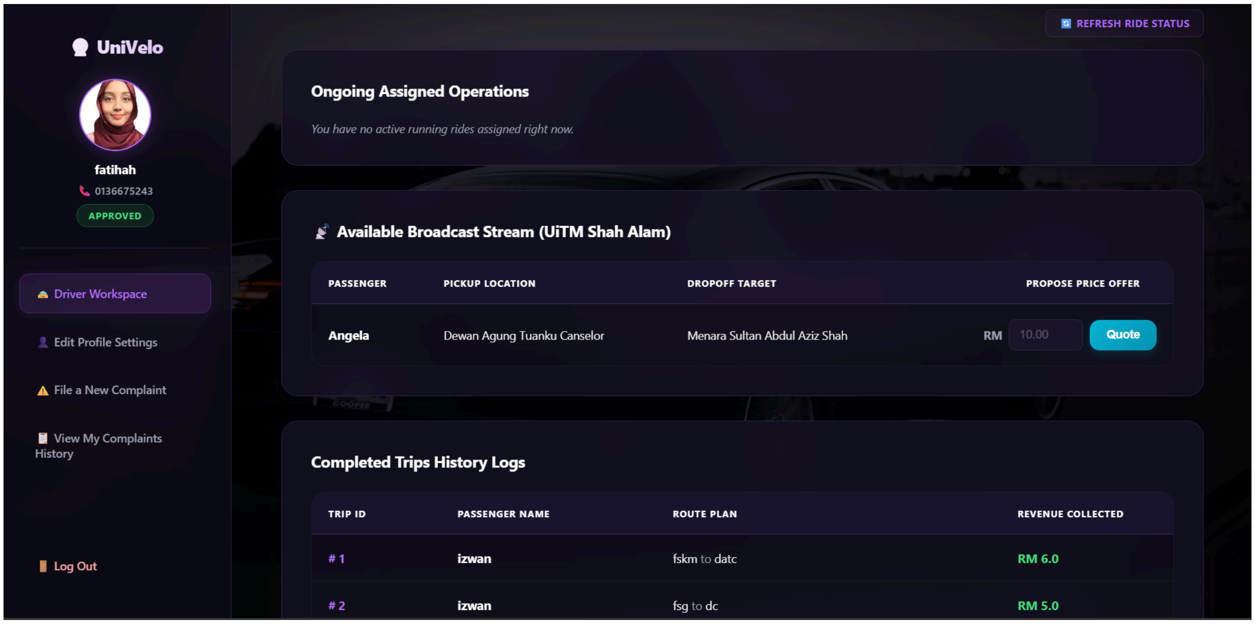

Figure 6.3.3 Driver Control Centre (Bidding)

Figure 6.3.3 shows when a passenger broadcasts a ride request, available drivers receive a mini-interface with the trip details and a price input field. Drivers can submit a bid to negotiate the fare. If the passenger rejects the initial offer, the driver can submit a revised bid.

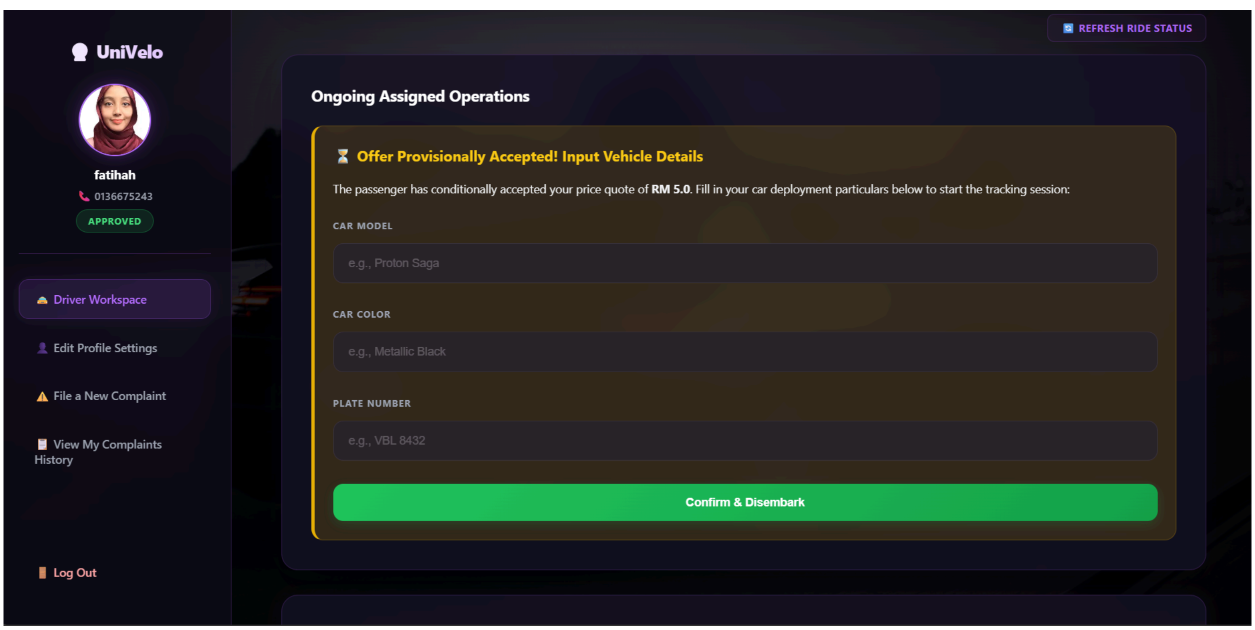

Figure 6.3.4 Driver Control Centre (Vehicle Information Entry)

Figure 6.3.4 shows once a passenger accepts the driver's fare, the driver is prompted to input their vehicle details, which are then shared with the passenger.

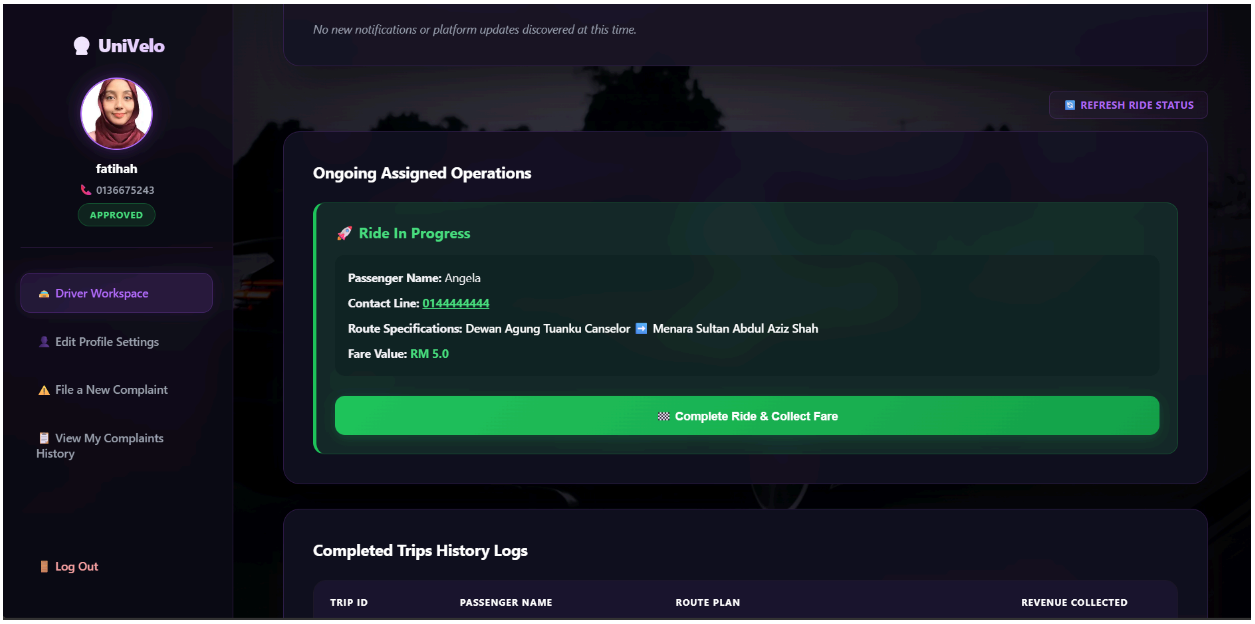

Figure 6.3.5 Driver Control Centre (Active Ride Tracking)

Figure 6.3.5 shows after vehicle details are submitted, the interface updates to an **In-Progress** state, displaying passenger details and trip info.

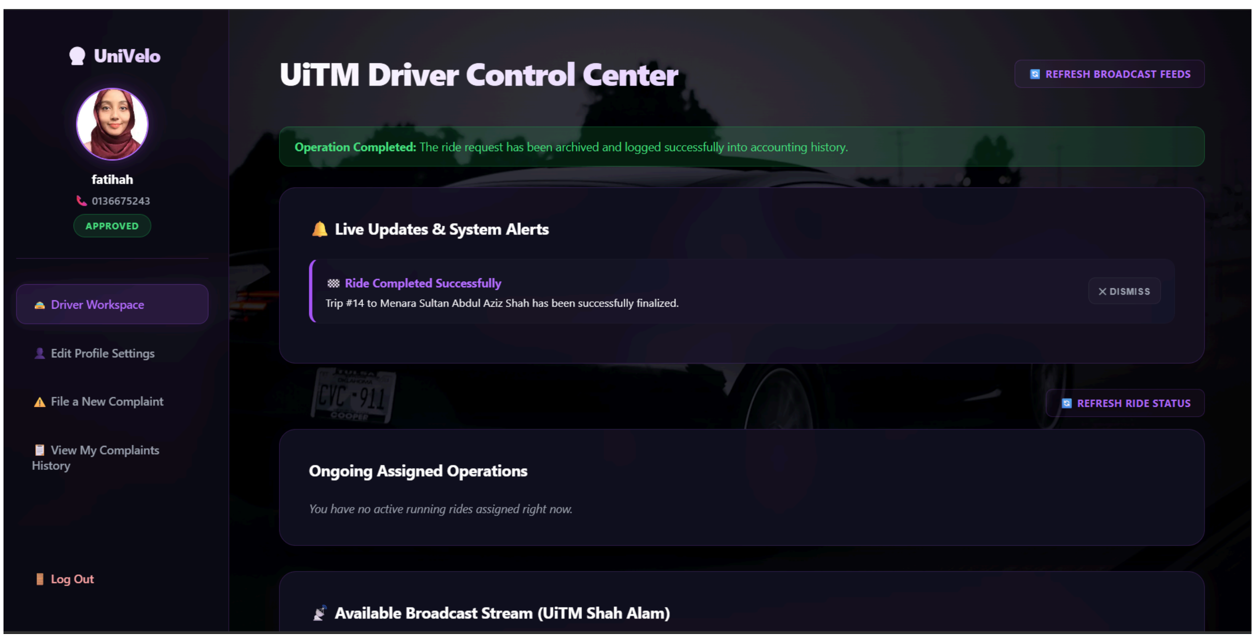

Figure 6.3.6 Driver Control Centre (Ride Completion Notification)

Figure 6.3.6 shows that upon reaching the destination, the driver clicks the **Complete** button. This action officially settles the trip, triggers a completion pop-up notification, and automatically saves the trip details to the **Ride History Logs**.

Figure 6.3.7 Driver Control Centre (Ride History Logs)

Figure 6.3.7 shows access to records of all completed trips.

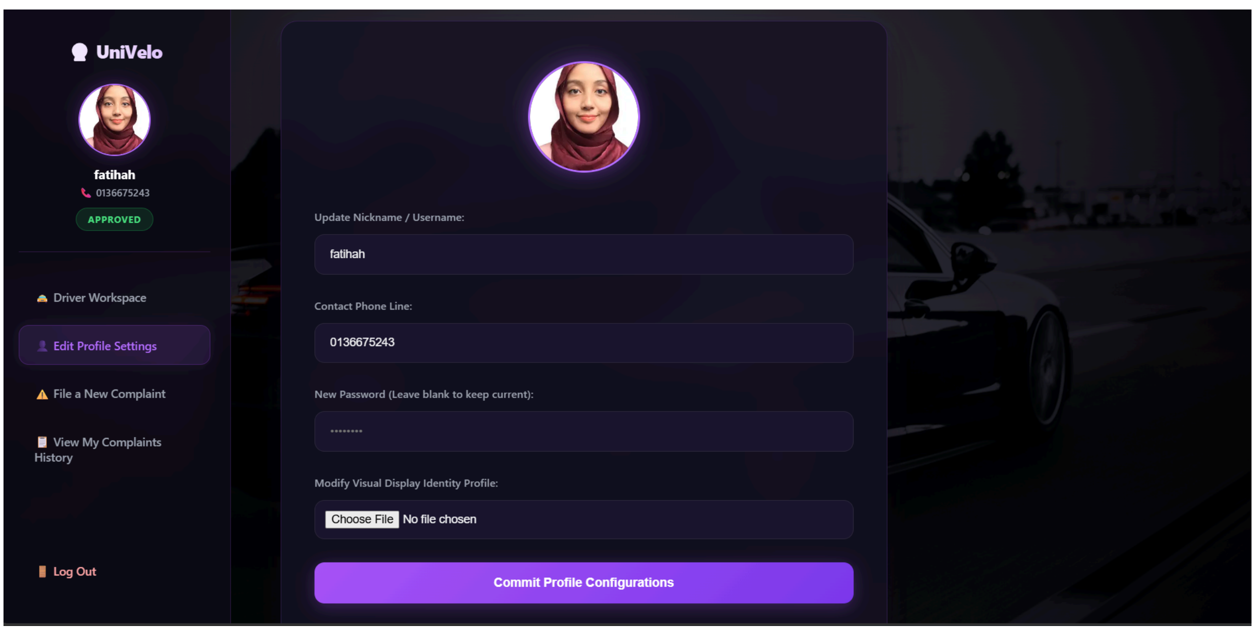

Figure 6.3.8 Edit Profile Settings

Figure 6.3.8 allows drivers to update their personal and account information.

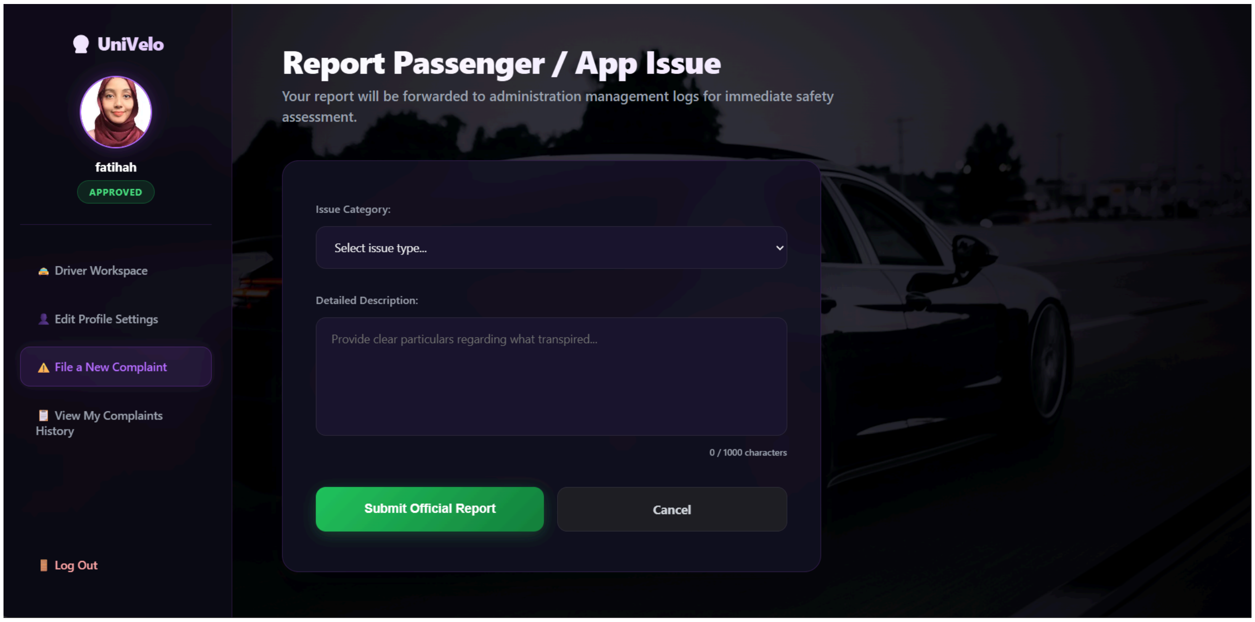

Figure 6.3.9 Report Passenger Page

Figure 6.3.9 shows that drivers can file formal complaints against issues.

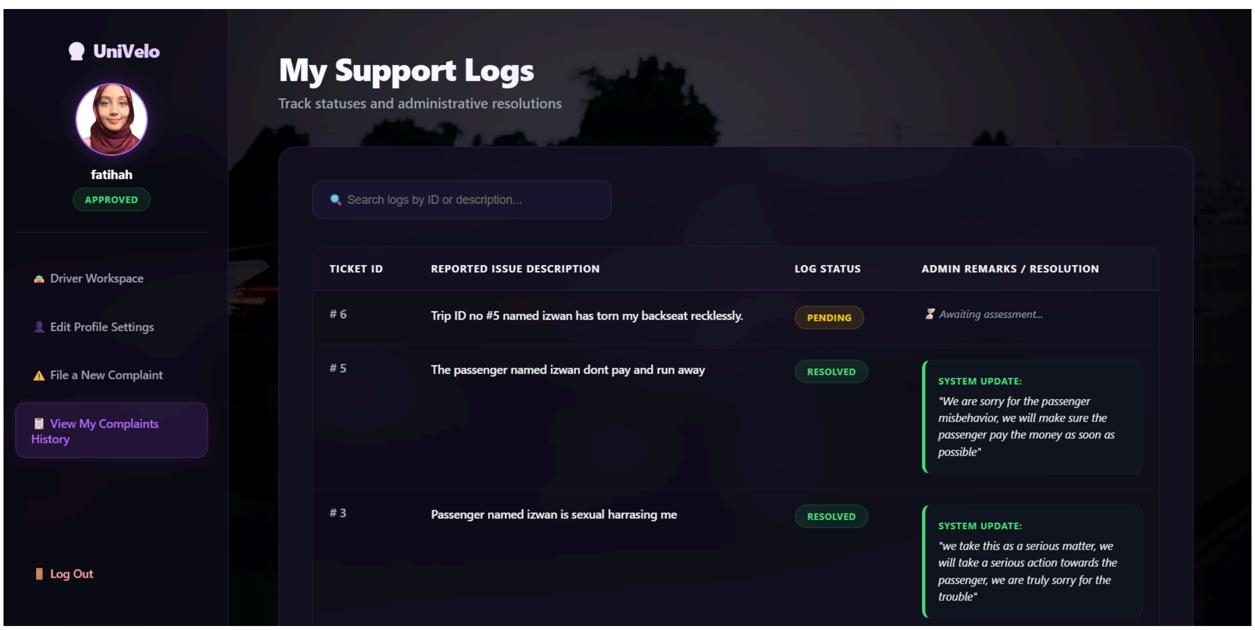

Figure 6.3.10 Complaint Logs Page

Figure 6.3.10 shows all pending and resolved complaints filed by the driver, alongside the Admin's resolution notes. Real-time notifications are sent to the driver as soon as an Admin resolves a complaint.

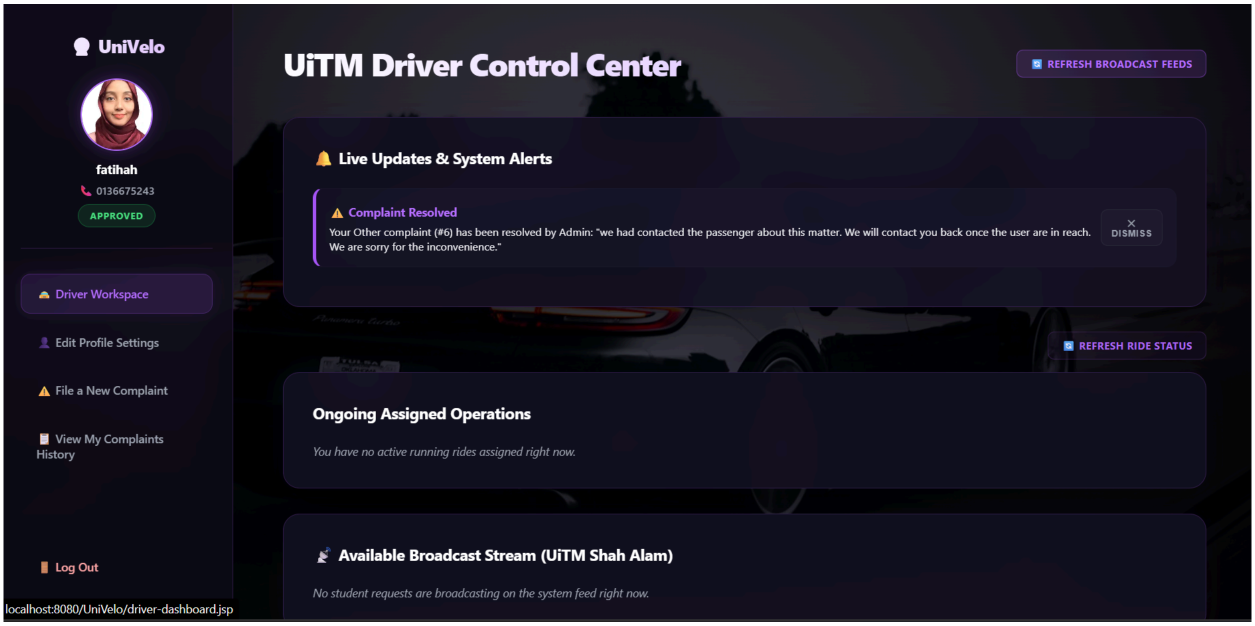

Figure 6.3.11 Driver Control Centre (Resolved Complaint Notification)

Figure 6.3.11 shows all pending and resolved complaints filed by the driver, alongside the Admin's resolution notes. Real-time notifications are sent to the driver as soon as an Admin resolves a complaint.

6.4 PASSENGER

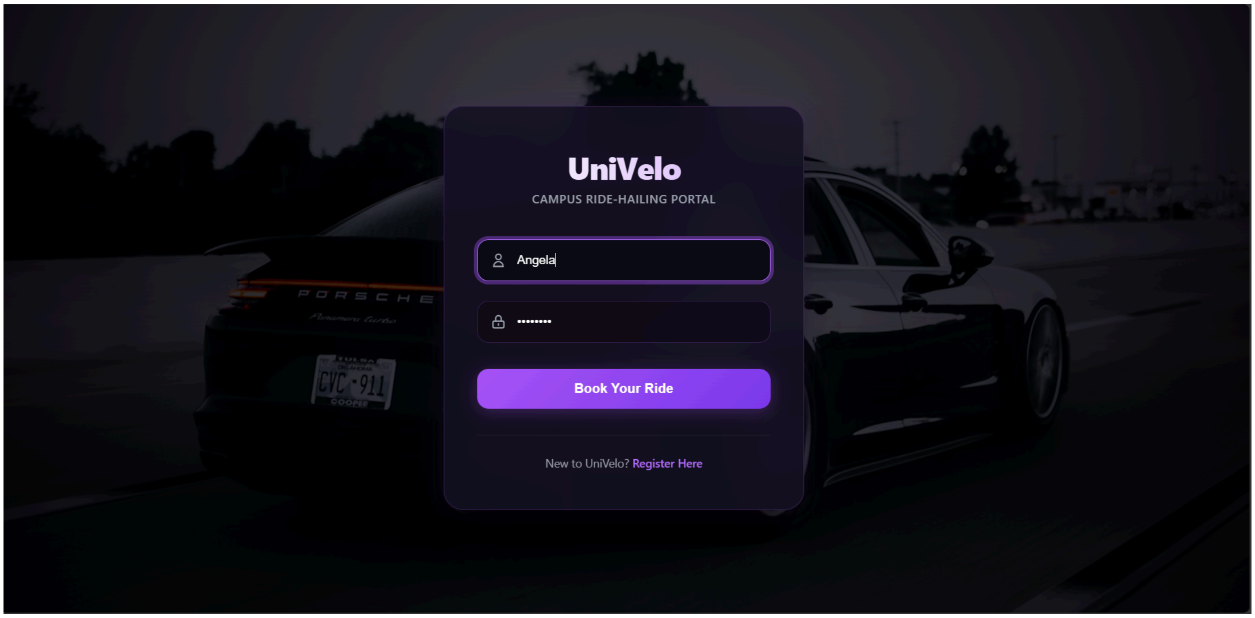

Figure 6.4.1 Login Page with Passenger's ID

Figure 6.4.1 shows a secure portal for approved passengers.

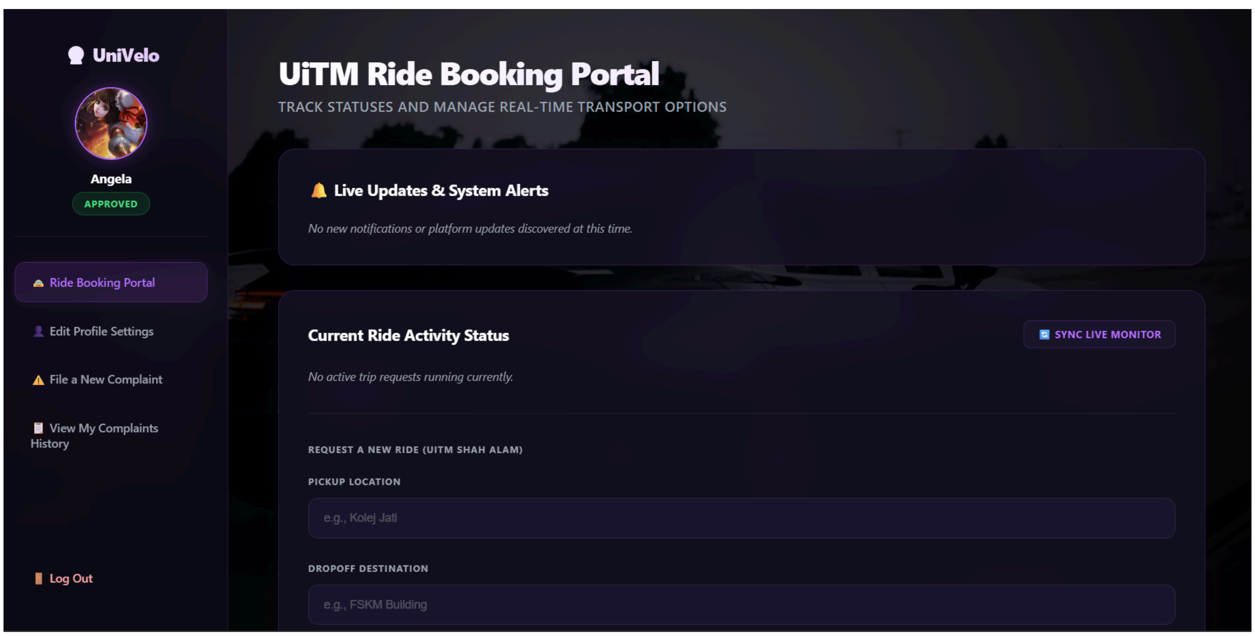

Figure 6.4.2 Login Page with Passenger's ID

Figure 6.4.2 shows the basic interface for drivers.

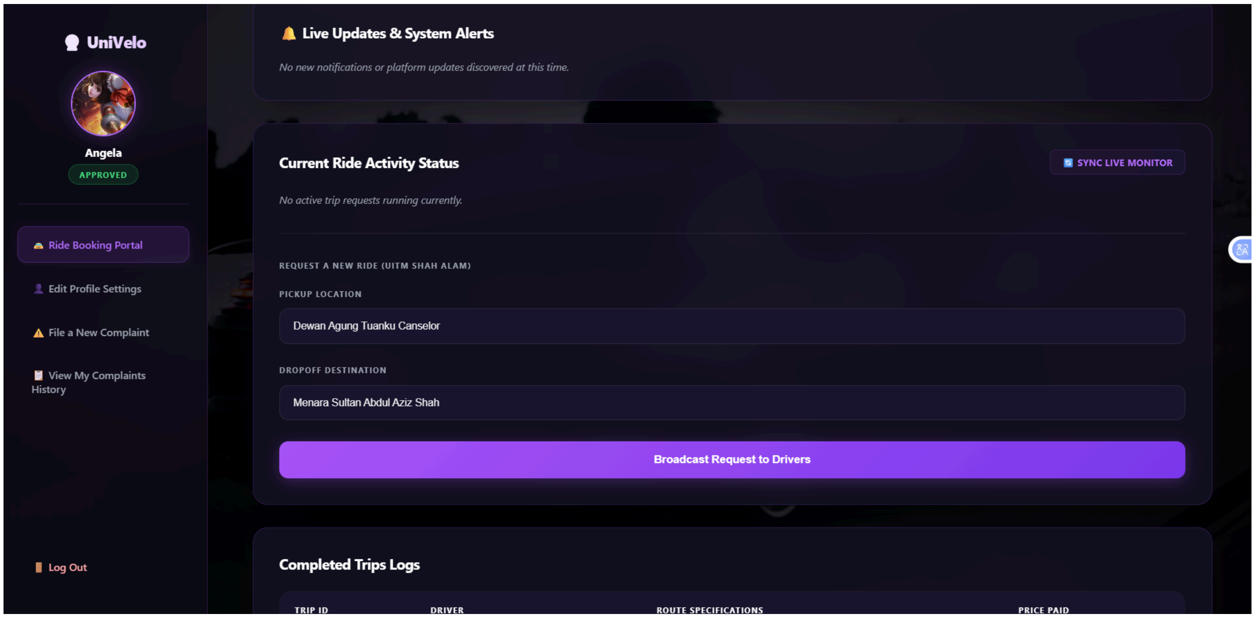

Figure 6.4.3 Ride Booking Portal (Ride Request Process)

Figure 6.4.3 shows when passengers enter their pickup and drop-off locations, then click **Broadcast Request** to send the trip details to all nearby available drivers.

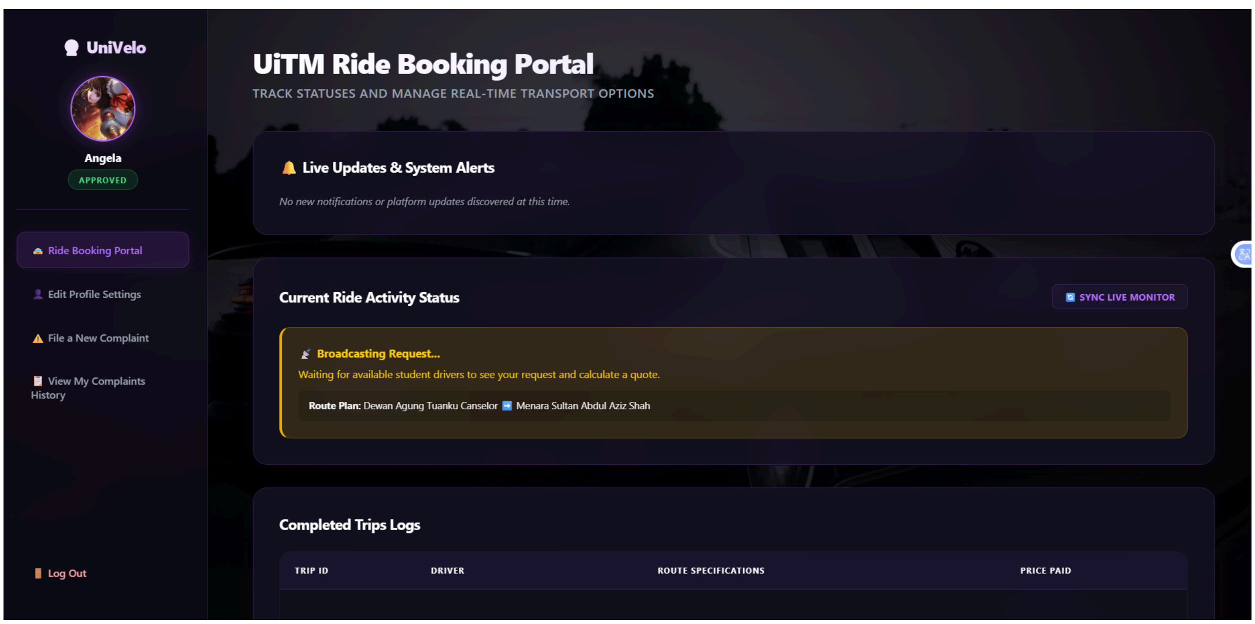

Figure 6.4.4 Ride Booking Portal (Pending Broadcast State)

Figure 6.4.4 shows a loading/waiting interface is displayed while the system searches for drivers and awaits initial bids.

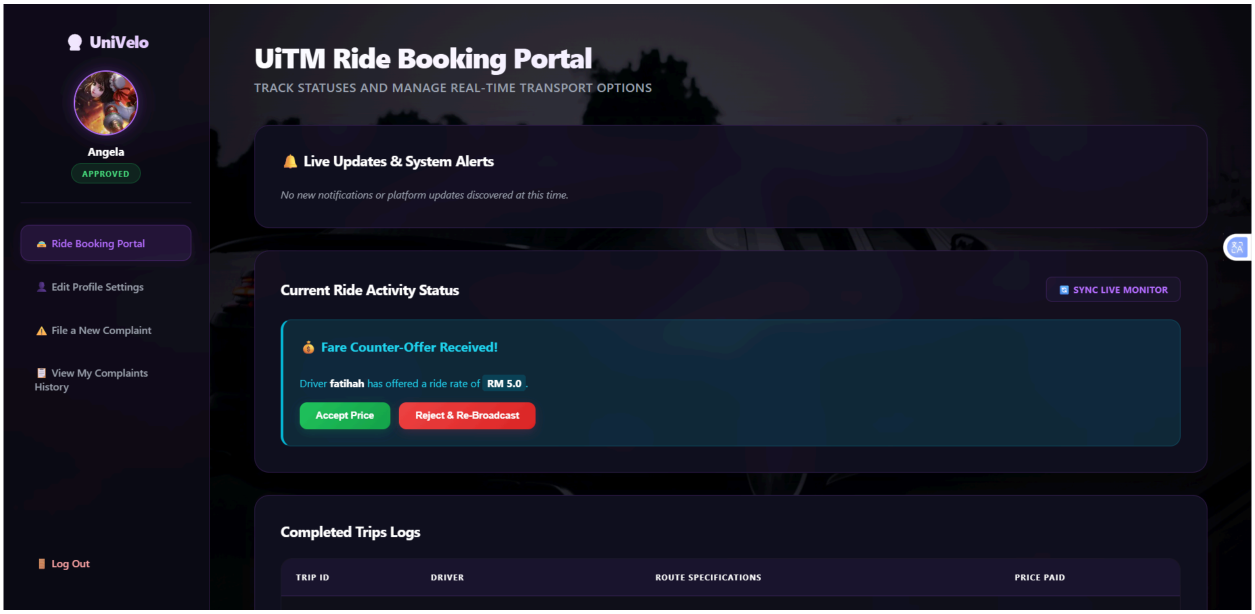

Figure 6.4.5 Ride Booking Portal (Fare Negotiation)

Figure 6.4.5 shows when a driver submits a bid, the passenger sees the offered price and can choose to **Accept** or **Reject** it. Rejecting the offer puts the request back into the active broadcast pool.

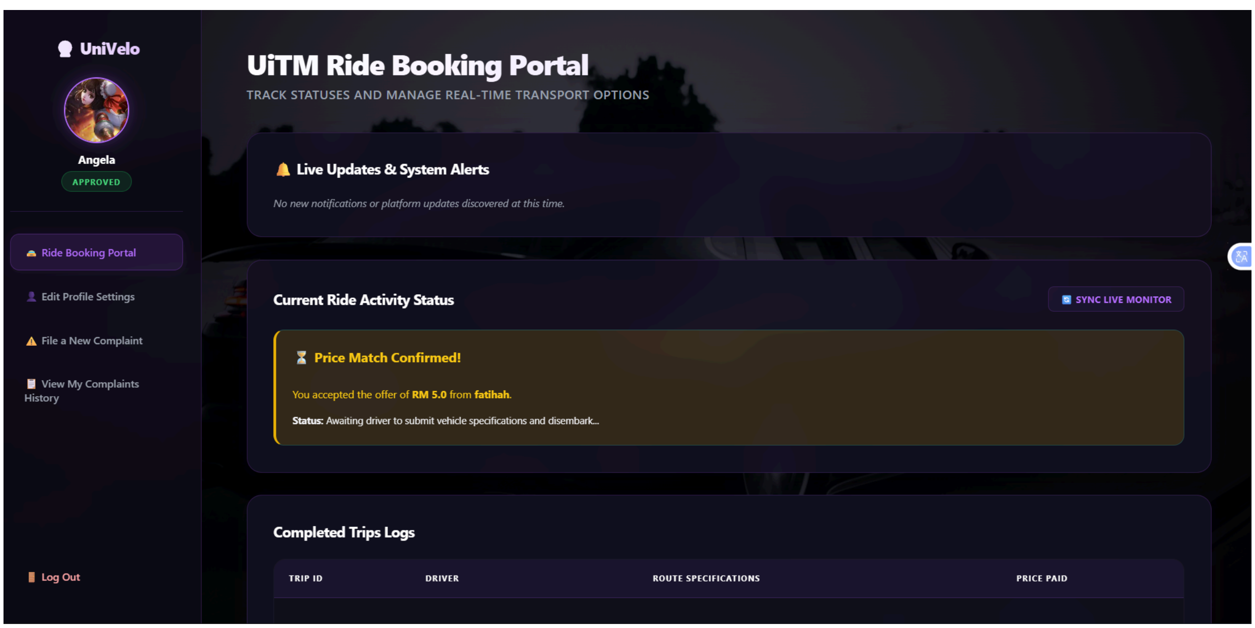

Figure 6.4.6 Ride Booking Portal (Ride Confirmation)

Figure 6.4.6 shows that once an offer is accepted, the status updates to **Confirmed**, and the passenger awaits the driver's vehicle information.

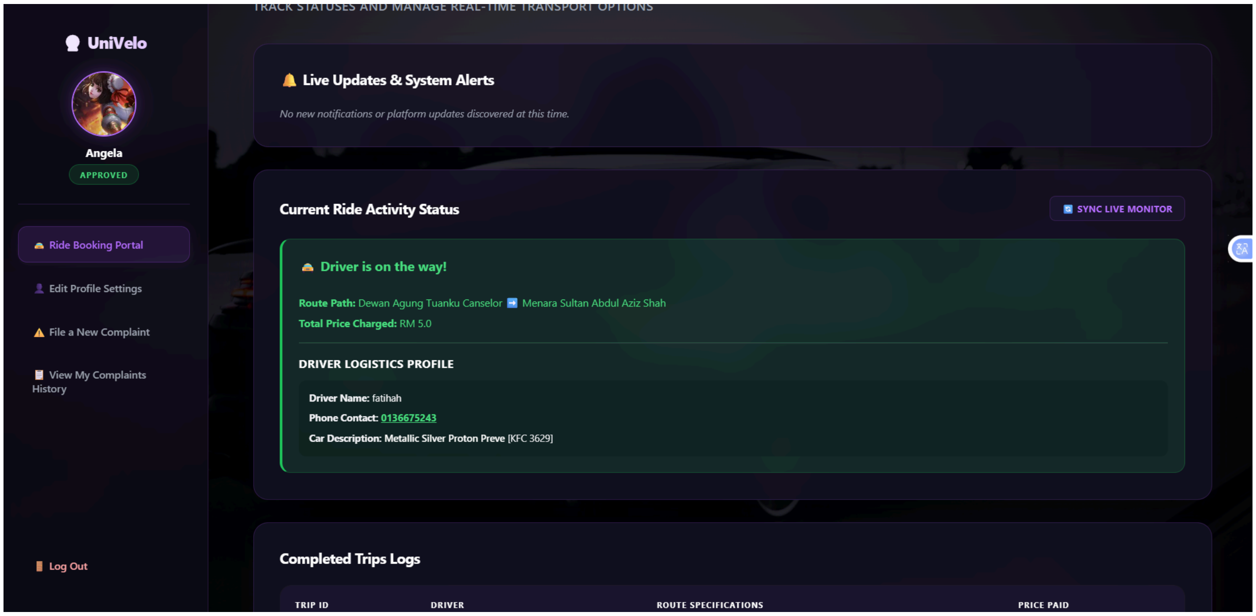

Figure 6.4.7 Ride Booking Portal (In-Progress Tracking)

Figure 6.4.7 shows that after the driver provides their vehicle details, a mini-interface displays the active trip status, driver profile, and vehicle information.

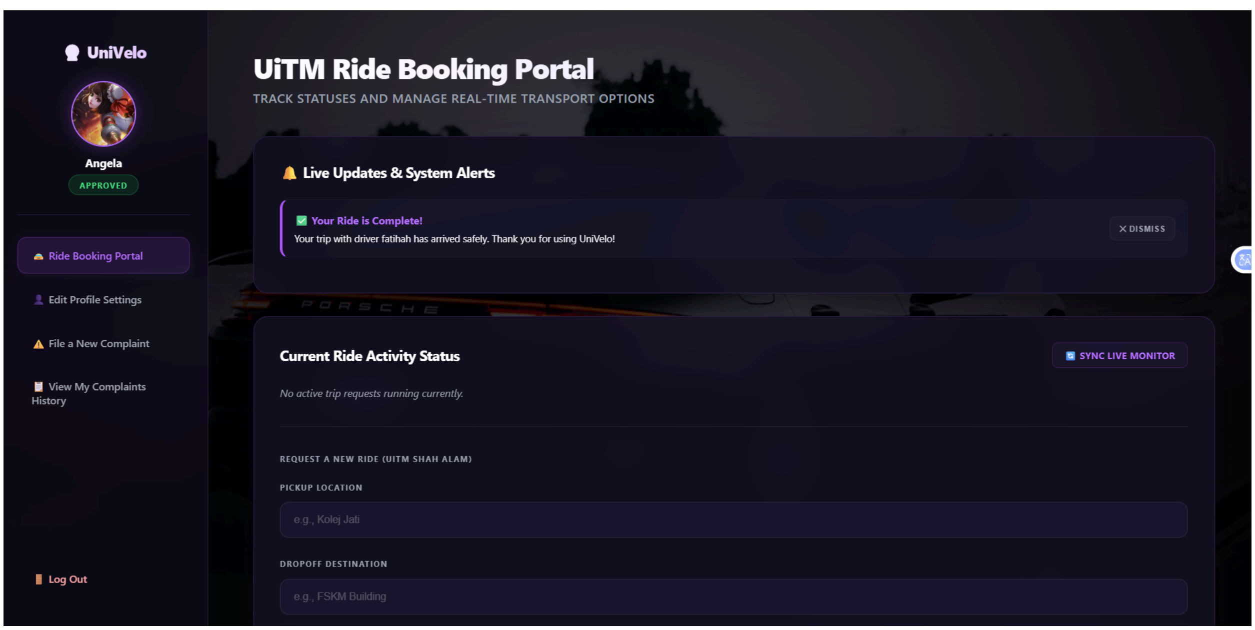

Figure 6.4.8 Ride Booking Portal (Ride Settlement)

Figure 6.4.8 shows that once the driver completes the trip, a notification pops up confirming the ride is finished, and the transaction is automatically saved to the passenger's history.

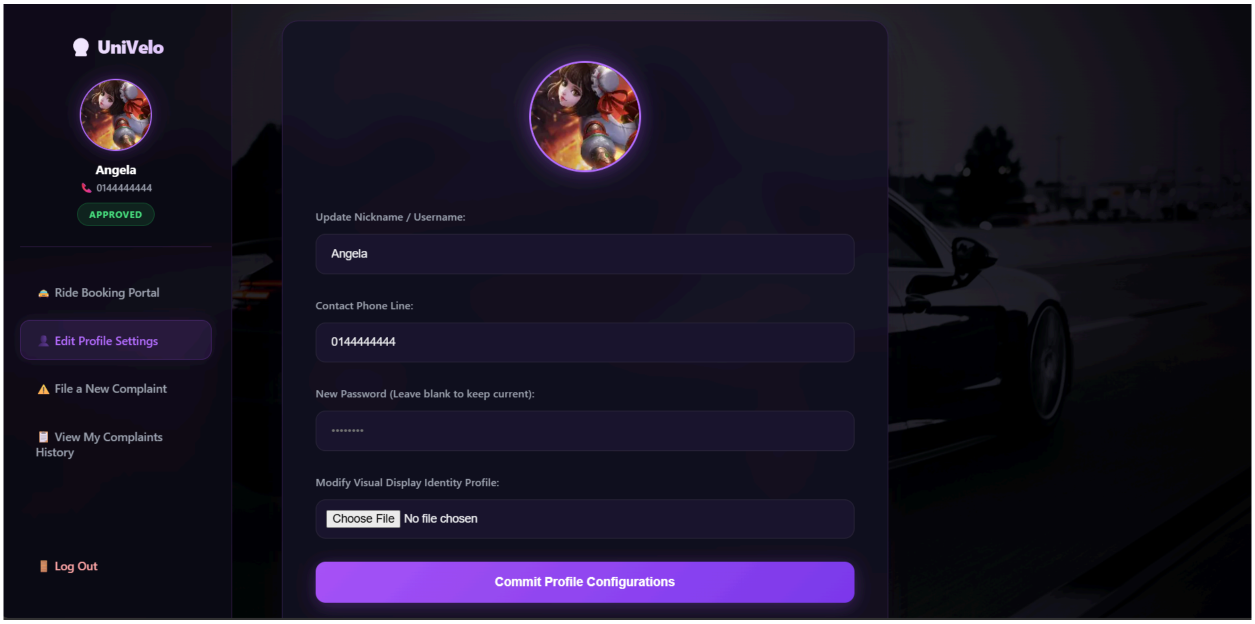

Figure 6.4.9 Passenger's Edit Profile Settings

Figure 6.4.9 allows passengers to manage their account details.

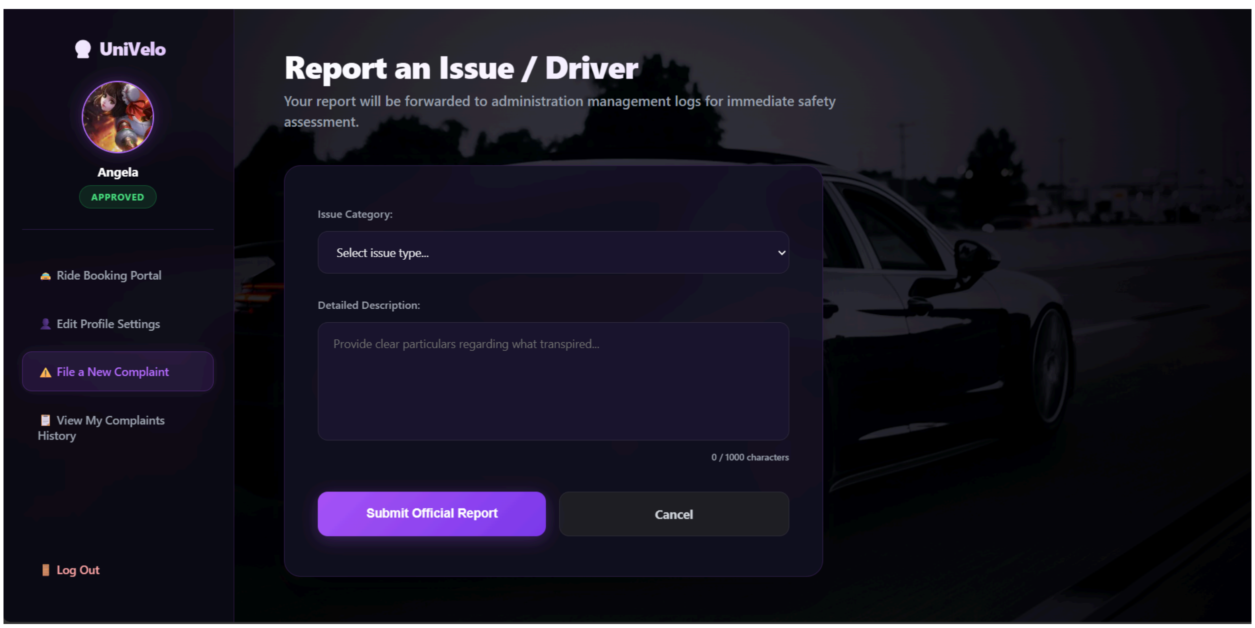

Figure 6.4.10 Passenger’s Report an Issue Page

Figure 6.4.10 shows a dedicated form to report issues or problematic behavior.

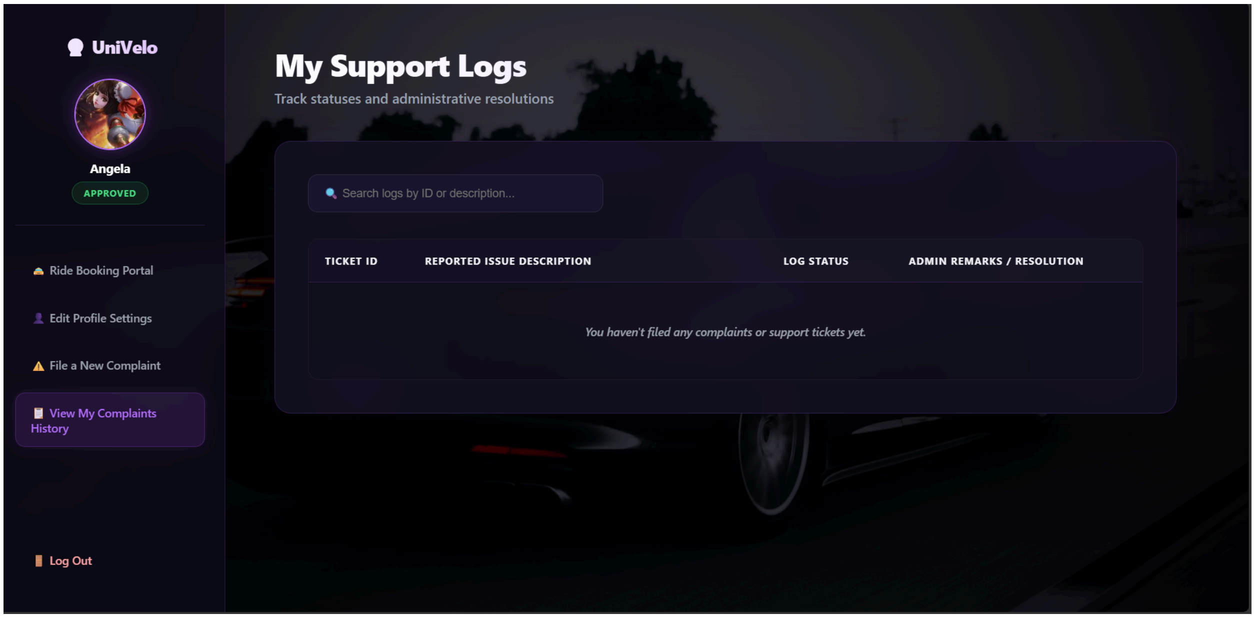

Figure 6.4.11 Passenger's Support Logs

Figure 6.4.11 shows a track the progress of filed complaints and view Admin resolution feedback.